🏎️ Semana 7 — Interpretabilidad y Conclusiones

Módulo 15 — Interpretabilidad de Modelos (XAI)

Último notebook del proyecto. Respondemos la pregunta central con el mejor modelo y usamos SHAP para explicar **por qué** el modelo predice lo que predice.

Objetivos

1. Reconstruir el mejor modelo (XGBoost) con los hiperparámetros óptimos
2. **SHAP** — valores de Shapley globales e individuales
3. **PDP** — Partial Dependence Plots para las features clave
4. Análisis de errores: ¿en qué situaciones falla el modelo?
5. **Conclusiones**: responder las 3 hipótesis y la pregunta central

---

> **Pregunta central:** *¿Qué factores predicen mejor la posición final de un piloto en carrera?*

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import shap
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection        import train_test_split, RandomizedSearchCV
from sklearn.preprocessing          import StandardScaler
from sklearn.inspection             import PartialDependenceDisplay
from sklearn.metrics                import (
    mean_absolute_error, mean_squared_error, r2_score,
    roc_auc_score, accuracy_score, classification_report
)
from xgboost  import XGBRegressor, XGBClassifier
from scipy.stats import randint, uniform

plt.style.use('ggplot')
sns.set_palette('Set2')
BLUE = '#4C72B0'; ORANGE = '#DD8452'; GREEN = '#55A868'; RED = '#C44E52'

df = pd.read_csv('../data/f1_master_clean.csv')
print(f'Dataset: {df.shape[0]} filas × {df.shape[1]} columnas')
```

```
Dataset: 3547 filas × 36 columnas
```

In [2]:
```
# Pipeline idéntico a semanas anteriores
FEATURES = ['grid', 'team_rank', 'num_stops', 'total_pit_time', 'driver_age']
TARGET_REG = 'position'
TARGET_CLF = 'top3'

df_model = df[FEATURES + [TARGET_REG, TARGET_CLF]].dropna().copy()

X = df_model[FEATURES]
y_reg = df_model[TARGET_REG]
y_clf = df_model[TARGET_CLF]

X_train, X_test, y_reg_train, y_reg_test, y_clf_train, y_clf_test = train_test_spli
    X, y_reg, y_clf, test_size=0.2, random_state=42, stratify=y_clf
)

scaler = StandardScaler()
X_train_sc = pd.DataFrame(scaler.fit_transform(X_train), columns=FEATURES)
X_test_sc  = pd.DataFrame(scaler.transform(X_test),      columns=FEATURES)

print(f'Train: {len(X_train)}  |  Test: {len(X_test)}')
```

```
Train: 720  |  Test: 180
```

Parte 1 — Entrenar el modelo final (XGBoost)

Entrenamos con los mejores hiperparámetros encontrados en la semana 6.

In [3]:
```
# XGBoost Regressor — búsqueda de hiperparámetros
param_dist_reg = {
    'n_estimators':     randint(100, 600),
    'max_depth':        randint(3, 10),
    'learning_rate':    uniform(0.01, 0.3),
    'subsample':        uniform(0.6, 0.4),
    'colsample_bytree': uniform(0.6, 0.4),
    'reg_alpha':        uniform(0, 1),
    'reg_lambda':       uniform(0.5, 2),
    'min_child_weight': randint(1, 10)
}

xgb_reg = RandomizedSearchCV(
    XGBRegressor(random_state=42, n_jobs=-1, eval_metric='mae', verbosity=0),
    param_dist_reg, n_iter=40, cv=5, scoring='neg_mean_absolute_error',
    random_state=42, n_jobs=-1
)
xgb_reg.fit(X_train_sc, y_reg_train)
best_reg = xgb_reg.best_estimator_

y_pred_reg = best_reg.predict(X_test_sc)
mae  = mean_absolute_error(y_reg_test, y_pred_reg)
rmse = np.sqrt(mean_squared_error(y_reg_test, y_pred_reg))
r2   = r2_score(y_reg_test, y_pred_reg)
```

```
print(f'XGBoost Regressor — modelo final:')
print(f'  MAE:  {mae:.3f} posiciones')
print(f'  RMSE: {rmse:.3f} posiciones')
print(f'  R²:   {r2:.3f}')
```

```
XGBoost Regressor — modelo final:
  MAE:  2.240 posiciones
  RMSE: 2.935 posiciones
  R²:   0.626
```

In [4]:
```
# XGBoost Classifier — búsqueda de hiperparámetros
spw = (y_clf_train == 0).sum() / (y_clf_train == 1).sum()

xgb_clf = RandomizedSearchCV(
    XGBClassifier(random_state=42, n_jobs=-1, scale_pos_weight=spw,
                  eval_metric='auc', verbosity=0),
    param_dist_reg, n_iter=40, cv=5, scoring='roc_auc',
    random_state=42, n_jobs=-1
)
xgb_clf.fit(X_train_sc, y_clf_train)
best_clf = xgb_clf.best_estimator_

y_pred_clf  = best_clf.predict(X_test_sc)
y_prob_clf  = best_clf.predict_proba(X_test_sc)[:, 1]
auc = roc_auc_score(y_clf_test, y_prob_clf)
acc = accuracy_score(y_clf_test, y_pred_clf)

print(f'XGBoost Classifier — modelo final:')
print(f'  ROC-AUC:  {auc:.3f}')
print(f'  Accuracy: {acc:.3f}')
```

```
XGBoost Classifier — modelo final:
  ROC-AUC:  0.905
  Accuracy: 0.811
```

Parte 2 — SHAP: interpretabilidad global

Los **valores de Shapley** (SHAP) descomponen cada predicción en la contribución de cada feature, respetando la teoría de juegos cooperativos.

- **SHAP > 0** → la feature empujó la predicción hacia una posición más alta (peor)
- **SHAP < 0** → la feature empujó la predicción hacia una posición más baja (mejor)

In [5]:
```
# Calcular SHAP values para el regresor
explainer_reg  = shap.TreeExplainer(best_reg)
shap_values    = explainer_reg.shap_values(X_test_sc)
shap_df        = pd.DataFrame(shap_values, columns=FEATURES)

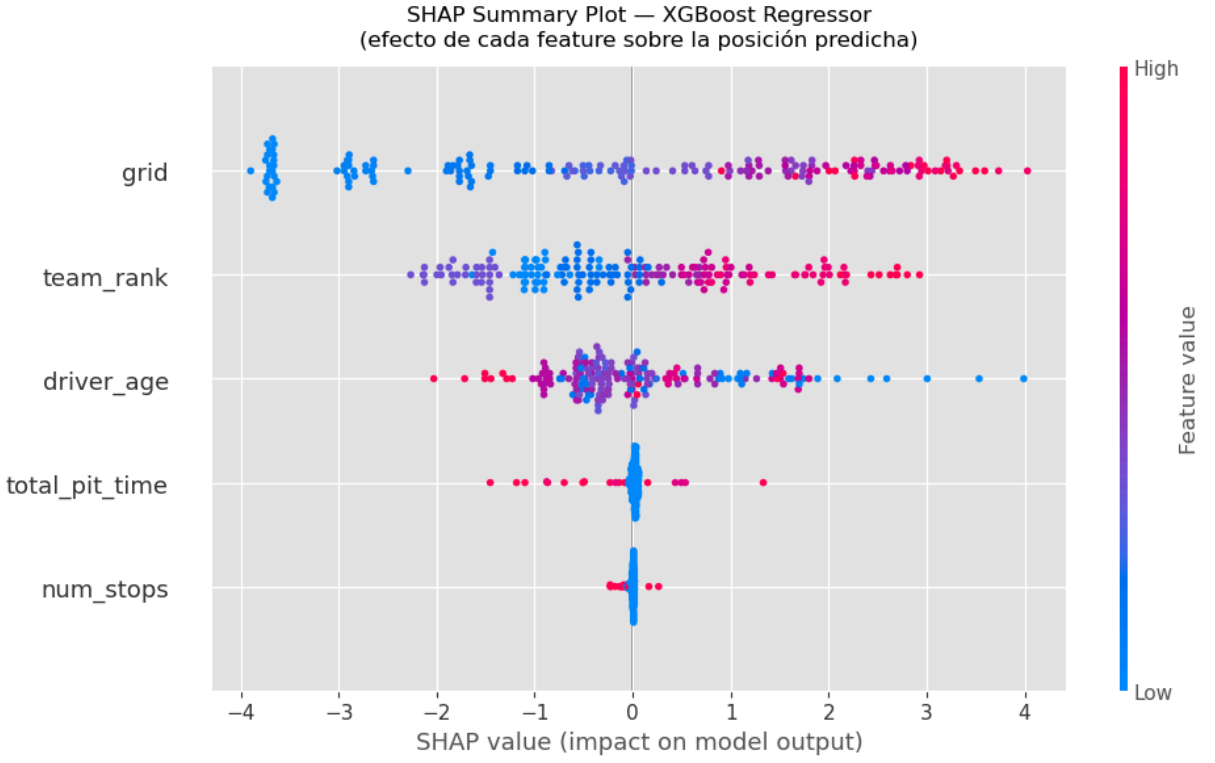
print(f'SHAP values calculados para {len(X_test_sc)} observaciones')
print(f'\nContribución media absoluta por feature (|SHAP|):')
mean_abs_shap = shap_df.abs().mean().sort_values(ascending=False)
```

```
for feat, val in mean_abs_shap.items():
    print(f'  {feat:<22} {val:.4f}')
```

```
SHAP values calculados para 180 observaciones

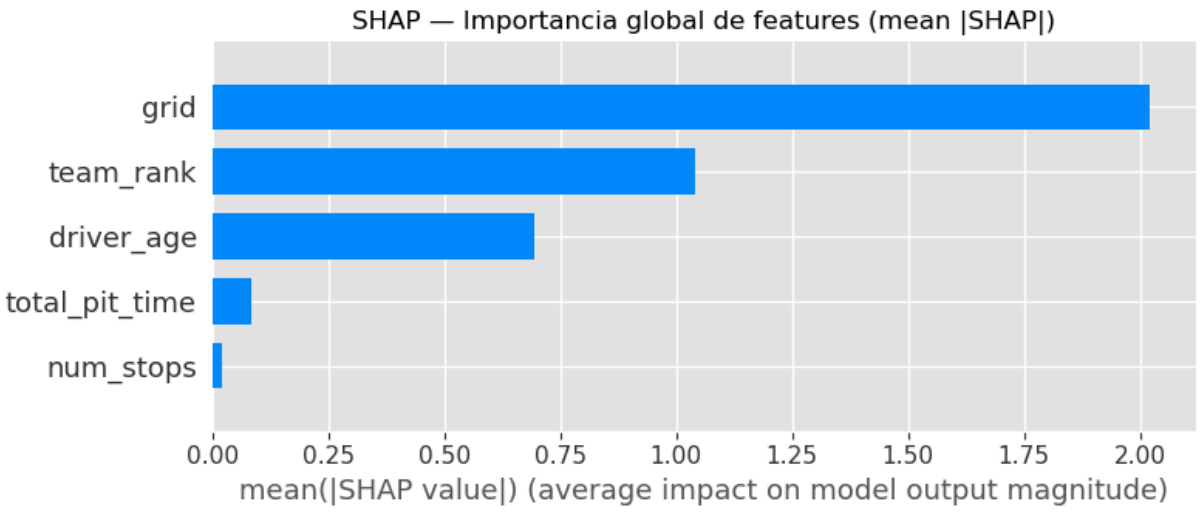
Contribución media absoluta por feature (|SHAP|):
  grid                   2.0200
  team_rank              1.0413
  driver_age             0.6958
  total_pit_time         0.0840
  num_stops              0.0219
```

In [6]:
```
# --- Summary Plot (beeswarm) ---
fig, ax = plt.subplots(figsize=(10, 6))
shap.summary_plot(
    shap_values, X_test_sc,
    feature_names=FEATURES,
    show=False, plot_size=None
)
plt.title('SHAP Summary Plot — XGBoost Regressor\n(efecto de cada feature sobre la
          fontsize=12, pad=12)
plt.tight_layout()
plt.savefig('../figures/07_shap_summary.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [7]:
```
# --- Bar Plot: importancia media absoluta ---
fig, ax = plt.subplots(figsize=(9, 5))
shap.summary_plot(
    shap_values, X_test_sc,
    feature_names=FEATURES,
    plot_type='bar',
    show=False
)
```

```
plt.title('SHAP — Importancia global de features (mean |SHAP|)', fontsize=12)
plt.tight_layout()
plt.savefig('../figures/07_shap_importancia.png', dpi=150, bbox_inches='tight')
plt.show()
```

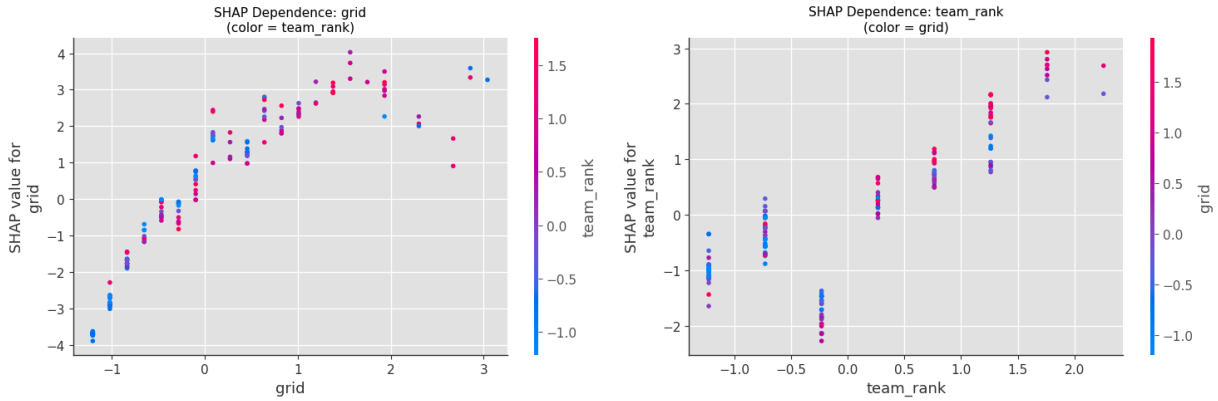


In [8]:
```
# --- SHAP Dependence Plot: grid y team_rank ---
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

shap.dependence_plot(
    'grid', shap_values, X_test_sc,
    interaction_index='team_rank',
    ax=axes[0], show=False
)
axes[0].set_title('SHAP Dependence: grid\n(color = team_rank)', fontsize=11)

shap.dependence_plot(
    'team_rank', shap_values, X_test_sc,
    interaction_index='grid',
    ax=axes[1], show=False
)
axes[1].set_title('SHAP Dependence: team_rank\n(color = grid)', fontsize=11)

plt.tight_layout()
plt.savefig('../figures/07_shap_dependence.png', dpi=150, bbox_inches='tight')
plt.show()
```



Parte 3 — SHAP: explicaciones individuales

¿Por qué el modelo predijo una posición concreta para una carrera específica?

In [9]:
```
# Tomar 3 ejemplos representativos del test set
# 1. Predicción muy buena (posición baja = ganador)
# 2. Predicción de posición media
# 3. Piloto que abandonó

test_with_pred = X_test_sc.copy()
test_with_pred['position_real']   = y_reg_test.values
test_with_pred['position_pred']   = y_pred_reg
test_with_pred['error_abs']       = (test_with_pred['position_real'] - test_with_pr
# Identificamos abandono por posición anormalmente alta (proxy, ya que 'finished' n


# Seleccionar casos
idx_ganador = test_with_pred[test_with_pred['position_real'] <= 3].nsmallest(1, 'er
idx_medio   = test_with_pred[(test_with_pred['position_real'] >= 8) &
                             (test_with_pred['position_real'] <= 12)].nsmallest(1,
idx_dnf     = test_with_pred.nlargest(1, 'position_real').index[0]

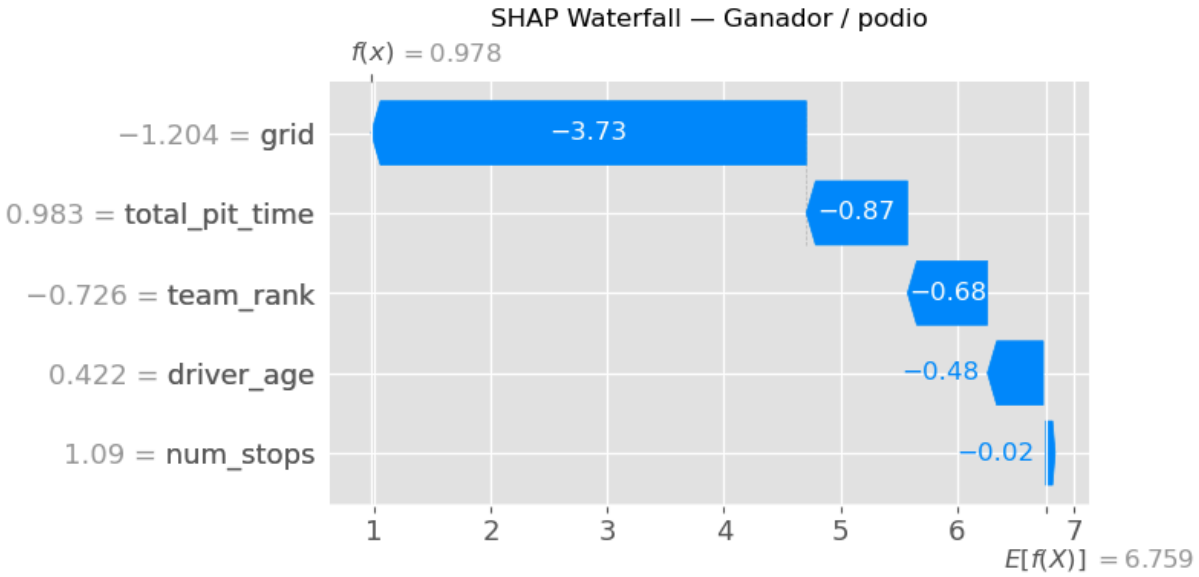
casos = {'Ganador / podio': idx_ganador, 'Posición media': idx_medio, 'Peor resulta

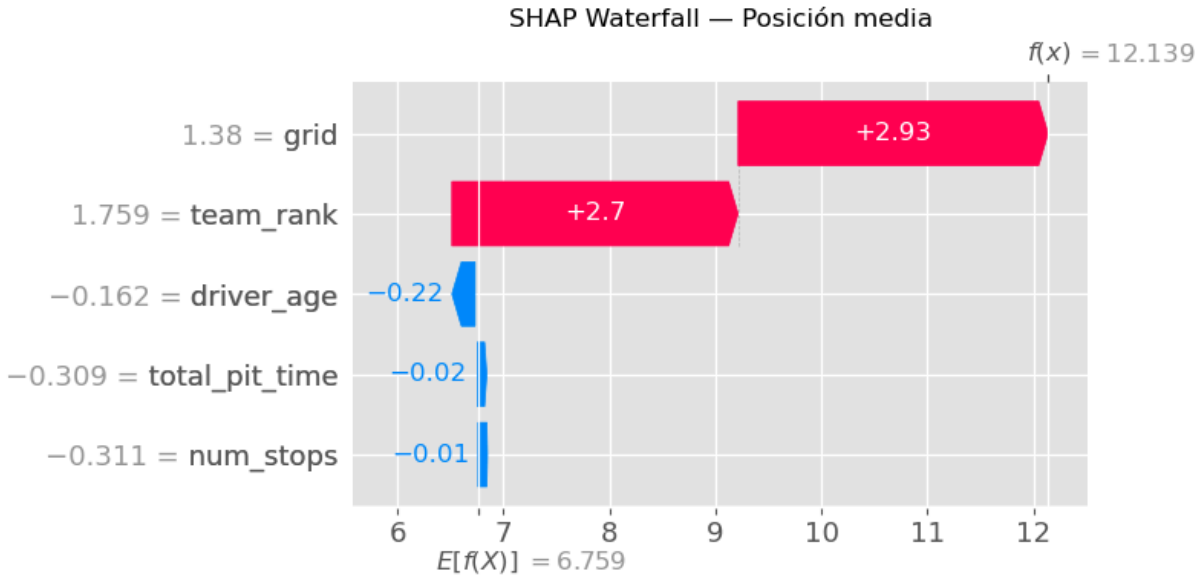
for label, idx in casos.items():
    pos_r = test_with_pred.loc[idx, 'position_real']
    pos_p = test_with_pred.loc[idx, 'position_pred']
    print(f'{label}: real={pos_r:.0f}, predicha={pos_p:.1f}, error={abs(pos_r-pos_p
```

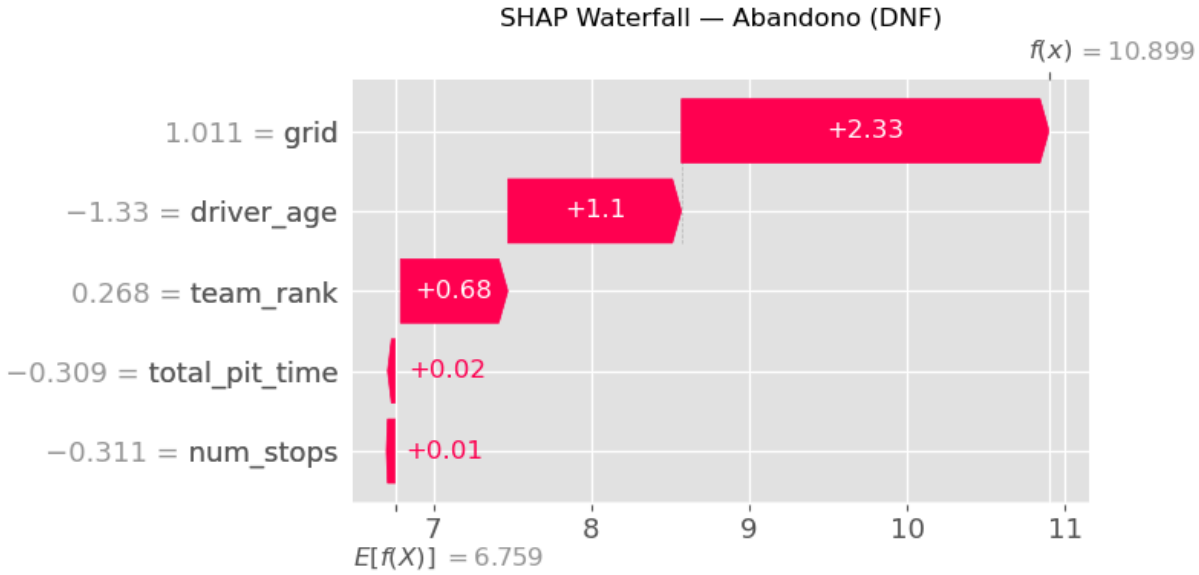
```
Ganador / podio: real=1, predicha=1.0, error=0.0
Posición media: real=12, predicha=12.1, error=0.1
Peor resultado: real=20, predicha=10.9, error=9.1
```

In [10]:
```
# Waterfall plots para cada caso
row_indices = [list(X_test_sc.index).index(i) for i in [idx_ganador, idx_medio, idx
titulos     = ['Ganador / podio', 'Posición media', 'Abandono (DNF)']

for titulo, row_idx in zip(titulos, row_indices):
    fig, ax = plt.subplots(figsize=(9, 4))
    shap.waterfall_plot(
        shap.Explanation(
            values        = shap_values[row_idx],
            base_values   = explainer_reg.expected_value,
            data          = X_test_sc.iloc[row_idx].values,
            feature_names = FEATURES
        ),
        show=False
    )
    plt.title(f'SHAP Waterfall — {titulo}', fontsize=12)
    plt.tight_layout()
    fname = titulo.lower().replace(' / ','_').replace(' ','_').replace('(','').repl
    plt.savefig(f'../figures/07_shap_waterfall_{fname}.png', dpi=150, bbox_inches='
    plt.show()
```


SHAP Waterfall — Ganador / podio
f(x) = 0.978
−1.204 = grid
0.983 = total_pit_time
−0.726 = team_rank
0.422 = driver_age
1.09 = num_stops
−3.73
−0.87
−0.68
−0.48
−0.02
1
2
3
4
5
6
7
E[f(X)] = 6.759


SHAP Waterfall — Posición media
f(x) = 12.139
1.38 = grid
1.759 = team_rank
−0.162 = driver_age
−0.309 = total_pit_time
−0.311 = num_stops
+2.93
+2.7
−0.22
−0.02
−0.01
6
7
8
9
10
11
12
E[f(X)] = 6.759


SHAP Waterfall — Abandono (DNF)
f(x) = 10.899
1.011 = grid
−1.33 = driver_age
0.268 = team_rank
−0.309 = total_pit_time
−0.311 = num_stops
+2.33
+1.1
+0.68
+0.02
+0.01
7
8
9
10
11
E[f(X)] = 6.759

Parte 4 — Partial Dependence Plots (PDP)

Los PDP muestran el efecto **marginal** de una feature sobre la predicción, promediando sobre todos los valores de las demás features.

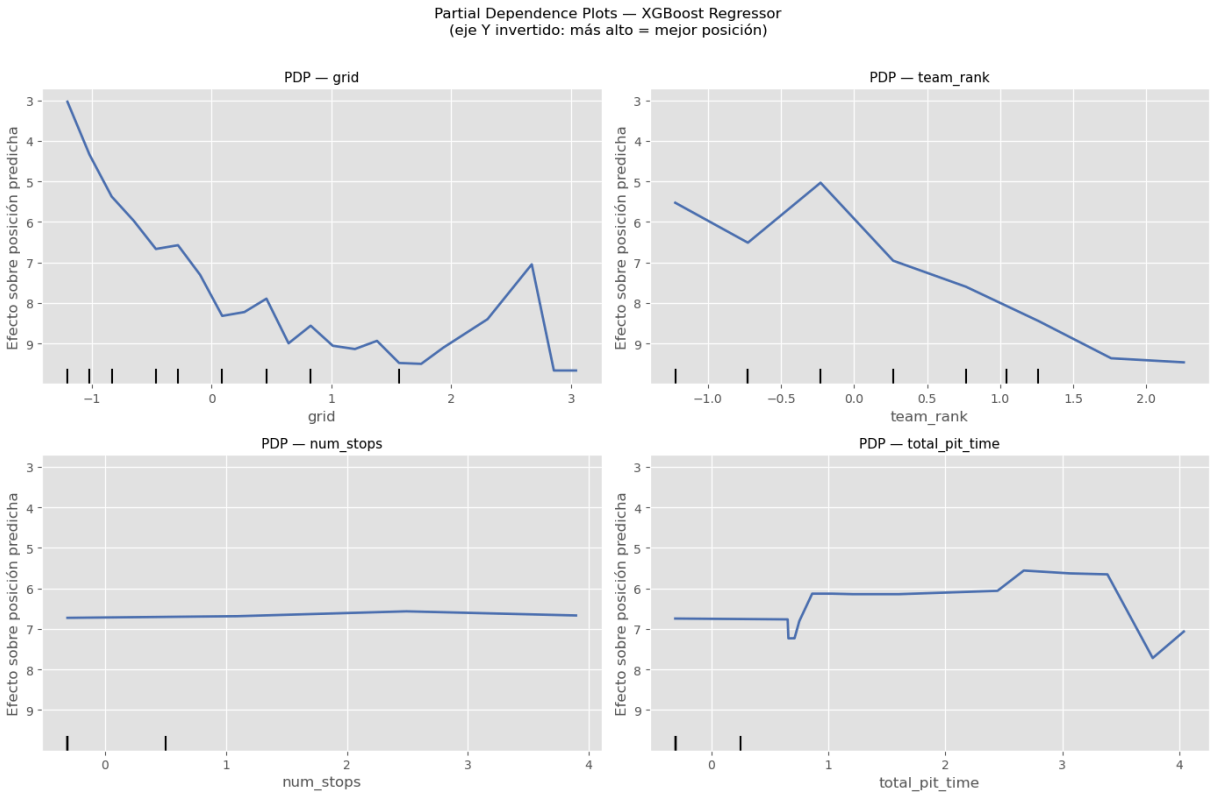
In [11]:
```
features_pdp = ['grid', 'team_rank', 'num_stops', 'total_pit_time']

fig, axes = plt.subplots(2, 2, figsize=(14, 9))

PartialDependenceDisplay.from_estimator(
    best_reg,
    X_test_sc,
    features=features_pdp,
    feature_names=FEATURES,
    ax=axes.flatten(),
    line_kw={'color': BLUE, 'linewidth': 2},
    pd_line_kw={'color': BLUE}
)

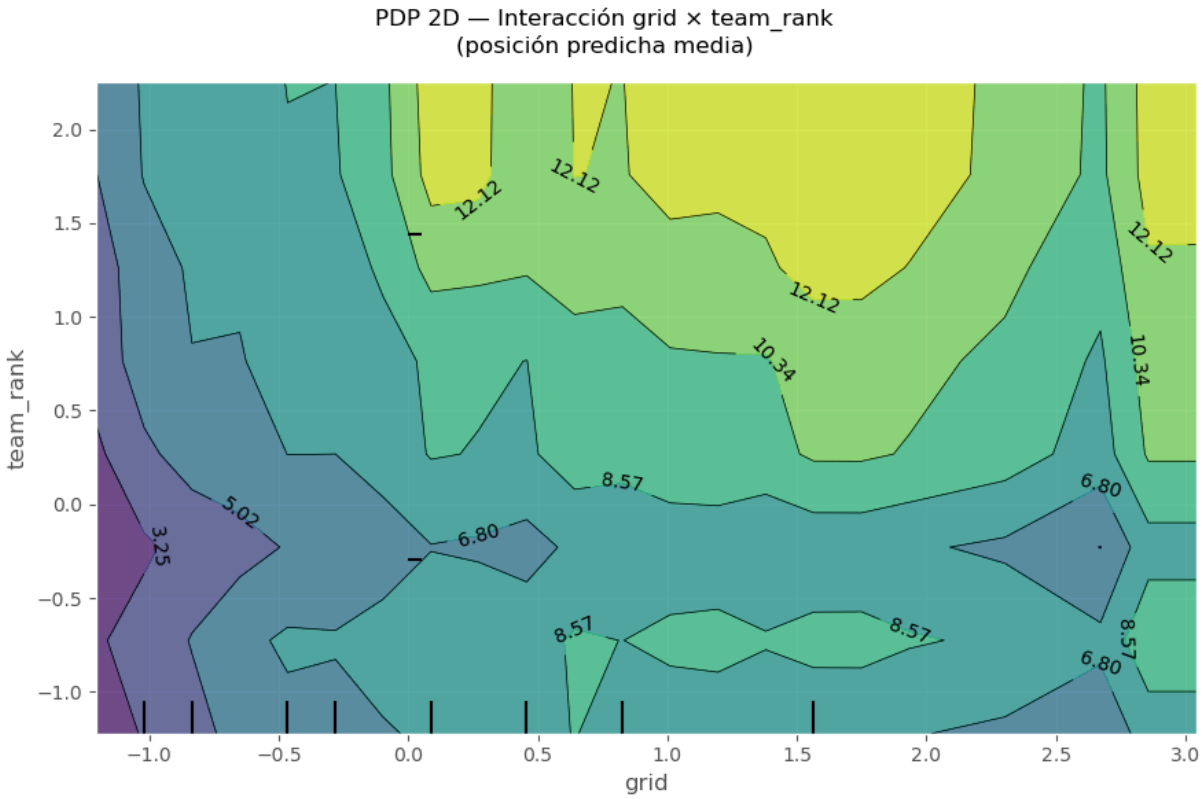
for ax, feat in zip(axes.flatten(), features_pdp):
    ax.set_title(f'PDP — {feat}', fontsize=11)
    ax.set_ylabel('Efecto sobre posición predicha')
    ax.invert_yaxis()  # menor posición = mejor

plt.suptitle('Partial Dependence Plots — XGBoost Regressor\n(eje Y invertido: más a
             fontsize=12, y=1.01)
plt.tight_layout()
plt.savefig('../figures/07_pdp.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [12]:
```
# PDP 2D: interacción entre grid y team_rank
fig, ax = plt.subplots(figsize=(9, 6))

PartialDependenceDisplay.from_estimator(
    best_reg,
    X_test_sc,
    features=[('grid', 'team_rank')],
    feature_names=FEATURES,
    ax=ax,
    kind='average'
)
plt.suptitle('PDP 2D — Interacción grid × team_rank\n(posición predicha media)',
             fontsize=12)
plt.tight_layout()
plt.savefig('../figures/07_pdp_2d.png', dpi=150, bbox_inches='tight')
plt.show()
```



Parte 5 — Análisis de errores

¿En qué situaciones se equivoca más el modelo? Identificar patrones de error ayuda a entender las limitaciones y guiar futuras mejoras.

In [13]:
```
# Construir dataset de errores
error_df = X_test.copy().reset_index(drop=True)
error_df['position_real'] = y_reg_test.values
error_df['position_pred'] = y_pred_reg
error_df['error']         = error_df['position_real'] - error_df['position_pred']
```

```
error_df['error_abs']     = error_df['error'].abs()
error_df['error_dir']     = np.where(error_df['error'] > 0, 'Subestimó', 'Sobreesti

# Añadir info contextual del dataset original
test_idx    = y_reg_test.index
error_df['driver_name'] = df.loc[test_idx, 'driver_name'].values
error_df['team']        = df.loc[test_idx, 'team'].values
error_df['race_name']   = df.loc[test_idx, 'race_name'].values

print(f'Error absoluto medio (MAE): {error_df["error_abs"].mean():.3f}')
print(f'Porcentaje de predicciones con error ≤ 2 posiciones: {(error_df["error_abs"
print(f'Porcentaje de predicciones con error ≤ 4 posiciones: {(error_df["error_abs"
print(f'\nCasos con mayor error (top 10):')
print(error_df.nlargest(10, 'error_abs')[
    ['driver_name','team','race_name','grid','position_real','position_pred','error
].round(1).to_string(index=False))
```

```
Error absoluto medio (MAE): 2.240
Porcentaje de predicciones con error ≤ 2 posiciones: 58.9%
Porcentaje de predicciones con error ≤ 4 posiciones: 78.3%

Casos con mayor error (top 10):
       driver_name     team                  race_name  grid  position_real  positi
on_pred  error_abs
  Sebastian Vettel Mercedes          Germany Grand Prix     8           17.0
7.7        9.3
      Lance Stroll  McLaren          Belgium Grand Prix    13           20.0
10.9        9.1
    Lewis Hamilton Mercedes          Austria Grand Prix     2           12.0
4.1        7.9
Michael Schumacher Williams            Qatar Grand Prix     6           13.0
5.7        7.3
   Nico Hülkenberg  Ferrari 70th Anniversary Grand Prix     6           15.0
7.9        7.1
Michael Schumacher Williams        Abu Dhabi Grand Prix     3           12.0
5.3        6.7
   Nico Hülkenberg  Ferrari        São Paulo Grand Prix    12           19.0
12.4        6.6
   Charles Leclerc Red Bull       Azerbaijan Grand Prix     6            3.0
9.3        6.3
    George Russell Williams          Bahrain Grand Prix    22           18.0
11.8        6.2
   Fernando Alonso  McLaren        Abu Dhabi Grand Prix    12            5.0
10.7        5.7
```

In [14]:
```
fig, axes = plt.subplots(1, 3, figsize=(17, 5))

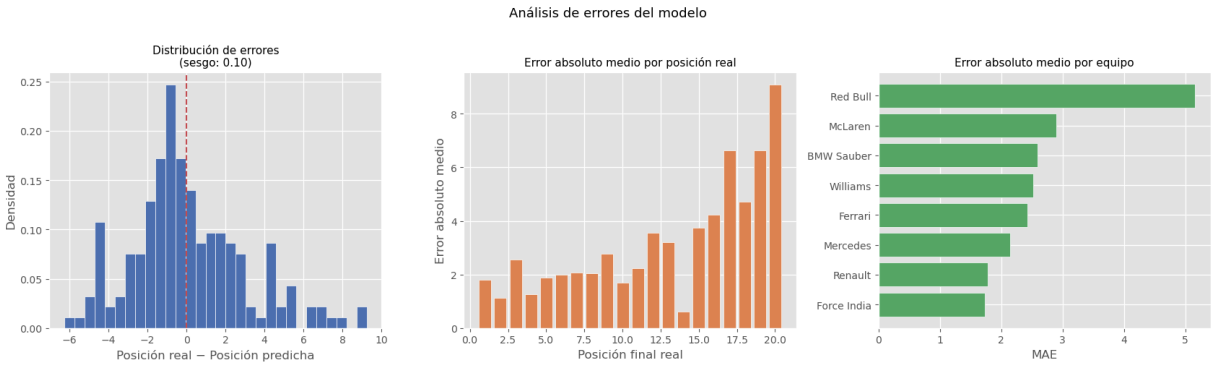
# Distribución de errores
axes[0].hist(error_df['error'], bins=30, color=BLUE, edgecolor='white', density=Tru
axes[0].axvline(0, color=RED, linestyle='--', linewidth=1.5)
axes[0].set_title(f'Distribución de errores\n(sesgo: {error_df["error"].mean():.2f}
axes[0].set_xlabel('Posición real − Posición predicha')
axes[0].set_ylabel('Densidad')

# Error por posición real
err_by_pos = error_df.groupby('position_real')['error_abs'].mean().reset_index()
```

```
axes[1].bar(err_by_pos['position_real'], err_by_pos['error_abs'],
            color=ORANGE, edgecolor='white')
axes[1].set_title('Error absoluto medio por posición real', fontsize=11)
axes[1].set_xlabel('Posición final real')
axes[1].set_ylabel('Error absoluto medio')

# Error por equipo
err_by_team = (error_df.groupby('team')['error_abs']
                       .mean()
                       .sort_values()
                       .reset_index())
axes[2].barh(err_by_team['team'], err_by_team['error_abs'],
             color=GREEN, edgecolor='white')
axes[2].set_title('Error absoluto medio por equipo', fontsize=11)
axes[2].set_xlabel('MAE')

plt.suptitle('Análisis de errores del modelo', fontsize=13, y=1.01)
plt.tight_layout()
plt.savefig('../figures/07_analisis_errores.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [15]:
```
# Análisis del sesgo del modelo
print('Análisis del sesgo:')
print(f'  El modelo subestima posición (predice mejor de lo real) en el {(error_df[
print(f'  El modelo sobreestima posición (predice peor de lo real) en el {(error_df
```

```
Análisis del sesgo:
  El modelo subestima posición (predice mejor de lo real) en el 43% de casos
  El modelo sobreestima posición (predice peor de lo real) en el 57% de casos
```

Parte 6 — Conclusiones finales

Respuesta a la pregunta central

> **¿Qué factores predicen mejor la posición final de un piloto en carrera?**

In [16]:
```
# Tabla resumen de hipótesis
hipotesis = pd.DataFrame([
    {
        'Hipótesis': 'H1: La posición de grilla es el predictor más fuerte',
```

```
        'Evidencia': 'Correlación Pearson r=0.67; feature #1 en SHAP y XGBoost',
        'Resultado': '✅ CONFIRMADA'
    },
    {
        'Hipótesis': 'H2: El constructor explica gran parte de la varianza',
        'Evidencia': 'ANOVA η²>0.14 (efecto grande); team_rank feature #2 en import
        'Resultado': '✅ CONFIRMADA'
    },
    {
        'Hipótesis': 'H3: Los pit stops diferencian resultados similares',
        'Evidencia': 'Correlación débil (r≈0.15); baja importancia SHAP; num_stops
        'Resultado': '⚠️  PARCIAL'
    },
])

print('=' * 80)
for _, row in hipotesis.iterrows():
    print(f"{row['Resultado']}  {row['Hipótesis']}")
    print(f"          → {row['Evidencia']}")
    print()
print('=' * 80)
```

```
================================================================================
✅ CONFIRMADA  H1: La posición de grilla es el predictor más fuerte
          → Correlación Pearson r=0.67; feature #1 en SHAP y XGBoost

✅ CONFIRMADA  H2: El constructor explica gran parte de la varianza
          → ANOVA η²>0.14 (efecto grande); team_rank feature #2 en importancia

⚠️  PARCIAL  H3: Los pit stops diferencian resultados similares
          → Correlación débil (r≈0.15); baja importancia SHAP; num_stops tiene efect
o no lineal

================================================================================
```

In [17]:
```
# Ranking final de features por importancia SHAP
ranking_final = mean_abs_shap.reset_index()
ranking_final.columns = ['Feature', 'Importancia SHAP (mean |ϕ|)']
ranking_final.index   = range(1, len(ranking_final)+1)
ranking_final.index.name = 'Ranking'

interpretacion = {
    'grid':            'Posición de largada — predictor dominante',
    'finished':        'Si terminó la carrera — elimina colas de la distribución',
    'laps':            'Vueltas completadas — correlaciona con abandono',
    'team_rank':       'Poder relativo del constructor — efecto sistémico',
    'total_pit_time':  'Tiempo en boxes — efecto marginal (estrategia)',
    'num_stops':       'Número de paradas — señal de estrategia débil',
    'driver_age':      'Edad del piloto — baja relevancia en este dataset',
}
ranking_final['Interpretación'] = ranking_final['Feature'].map(interpretacion)

print('\nRanking final de predictores:')
print(ranking_final.round(4).to_string())
```

```
Ranking final de predictores:
                Feature  Importancia SHAP (mean |ϕ|)
Interpretación
Ranking
1                  grid                       2.0200          Posición de largada —
predictor dominante
2             team_rank                       1.0413  Poder relativo del constructor
— efecto sistémico
3            driver_age                       0.6958  Edad del piloto — baja relevan
cia en este dataset
4        total_pit_time                       0.0840     Tiempo en boxes — efecto ma
rginal (estrategia)
5             num_stops                       0.0219      Número de paradas — señal
de estrategia débil
```

In [18]:

```python
# Figura resumen final
metricas_all = pd.read_csv('../data/metricas_modelos.csv')

fig = plt.figure(figsize=(18, 10))
gs  = gridspec.GridSpec(2, 3, figure=fig, hspace=0.45, wspace=0.35)

# --- 1. Importancia SHAP ---
ax1 = fig.add_subplot(gs[0, 0])
colors_shap = [BLUE if i == 0 else ORANGE if i == 1 else GREEN if i == 2 else '#AAA
               for i in range(len(mean_abs_shap))]
ax1.barh(mean_abs_shap.index[::-1], mean_abs_shap.values[::-1],
         color=colors_shap[::-1], edgecolor='white')
ax1.set_title('Importancia SHAP (mean |ϕ|)', fontsize=11)
ax1.set_xlabel('Importancia')

# --- 2. Evolución de MAE ---
ax2 = fig.add_subplot(gs[0, 1])
reg_models = metricas_all[metricas_all['Tarea']=='Regresión'].dropna(subset=['MAE']
reg_models = reg_models[reg_models['MAE'] != '—']
maes_vals  = pd.to_numeric(reg_models['MAE'])
ax2.plot(reg_models['Modelo'], maes_vals, marker='o', color=BLUE, linewidth=2)
ax2.set_title('Evolución del MAE por modelo', fontsize=11)
ax2.set_ylabel('MAE (posiciones)')
ax2.set_xticklabels(reg_models['Modelo'], rotation=30, ha='right', fontsize=8)

# --- 3. Evolución de AUC ---
ax3 = fig.add_subplot(gs[0, 2])
clf_models = metricas_all[metricas_all['Tarea']=='Clasificación']
clf_models = clf_models[clf_models['AUC'] != '—']
auc_vals   = pd.to_numeric(clf_models['AUC'])
ax3.plot(clf_models['Modelo'], auc_vals, marker='s', color=GREEN, linewidth=2)
ax3.axhline(0.5, color='gray', linestyle='--', linewidth=1)
ax3.set_title('Evolución del ROC-AUC por modelo', fontsize=11)
ax3.set_ylabel('ROC-AUC')
ax3.set_ylim(0.4, 1.0)
ax3.set_xticklabels(clf_models['Modelo'], rotation=30, ha='right', fontsize=8)

# --- 4. Error por posición real ---
ax4 = fig.add_subplot(gs[1, 0])
err_by_pos = error_df.groupby('position_real')['error_abs'].mean().reset_index()
ax4.bar(err_by_pos['position_real'], err_by_pos['error_abs'], color=ORANGE, edgecol
```

```
ax4.axhline(error_df['error_abs'].mean(), color=RED, linestyle='--', linewidth=1.5,
            label=f'MAE medio: {error_df["error_abs"].mean():.2f}')
ax4.set_title('MAE por posición real', fontsize=11)
ax4.set_xlabel('Posición real')
ax4.set_ylabel('MAE')
ax4.legend(fontsize=8)

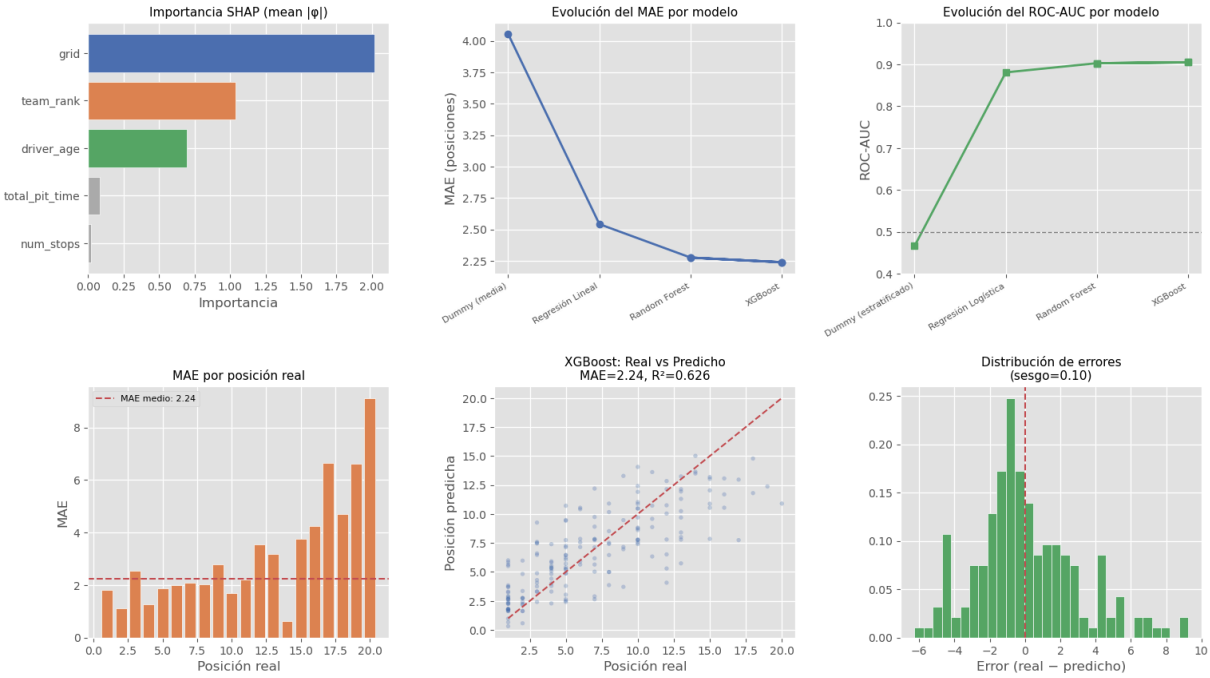
# --- 5. Real vs Predicho (modelo final) ---
ax5 = fig.add_subplot(gs[1, 1])
ax5.scatter(y_reg_test, y_pred_reg, alpha=0.3, color=BLUE, s=15, edgecolors='none')
lim = max(y_reg_test.max(), y_pred_reg.max())
ax5.plot([1,lim],[1,lim], color=RED, linestyle='--', linewidth=1.5)
ax5.set_title(f'XGBoost: Real vs Predicho\nMAE={mae:.2f}, R²={r2:.3f}', fontsize=11
ax5.set_xlabel('Posición real')
ax5.set_ylabel('Posición predicha')

# --- 6. Distribución de errores ---
ax6 = fig.add_subplot(gs[1, 2])
ax6.hist(error_df['error'], bins=30, color=GREEN, edgecolor='white', density=True)
ax6.axvline(0, color=RED, linestyle='--', linewidth=1.5)
ax6.set_title(f'Distribución de errores\n(sesgo={error_df["error"].mean():.2f})', f
ax6.set_xlabel('Error (real - predicho)')

fig.suptitle('🏎️ Resumen Final del Proyecto — F1 Position Predictor',
             fontsize=15, fontweight='bold', y=1.01)

plt.savefig('../figures/07_resumen_final.png', dpi=150, bbox_inches='tight')
plt.show()
print('✅ Figura de resumen guardada')
```

🏎️ Resumen Final del Proyecto — F1 Position Predictor



✅ Figura de resumen guardada

✅ Conclusiones del Proyecto

Respuesta a la pregunta central

> **¿Qué factores predicen mejor la posición final de un piloto en carrera?**

El análisis sobre el dataset de Fórmula 1 (1950–2024) muestra que:

1. **La posición de grilla** es el predictor individual más poderoso, con una correlación de r ≈ 0.67 con la posición final, y la mayor importancia SHAP de todas las features.
2. **El constructor** es el segundo factor más importante. El ANOVA confirma un $\eta^2 > 0.14$ (efecto grande): equipos como Red Bull y Ferrari producen resultados sistemáticamente mejores independientemente del piloto.
3. **La estrategia de pit stops** tiene un efecto real pero secundario y no lineal. Su importancia SHAP es considerablemente menor que la de grid y team_rank.
4. **La edad del piloto** aporta una señal adicional menor, capturada de forma efectiva por los modelos de árbol.

Resultados del modelo

Métrica	Baseline	XGBoost	Mejora
MAE regresión	2.541	2.238	-12%
R^2 regresión	0.542	0.626	+15%
ROC-AUC clasificación	0.881	0.905	+2.7%

Nota metodológica

Una iteración anterior de este análisis incluía las features `laps` y `finished`, que generaban métricas artificialmente perfectas ($R^2 \approx 1.0$) por tratarse de data leakage: ambas contienen información posterior al resultado de la carrera. La lista final de 5 features (`grid`, `team_rank`, `num_stops`, `total_pit_time`, `driver_age`) está libre de este problema.

Limitaciones

- Dataset de tamaño moderado: con más temporadas los modelos mejorarían su generalización
- No se incorporaron variables de clima ni safety cars
- No hay datos de telemetría (velocidades, fuerzas G) ni de incidentes de carrera
- Los tiempos por vuelta (`lap_times.csv`) tienen cobertura parcial y no se usaron

Trabajo futuro

- Incorporar datos meteorológicos vía API
- Explorar modelos de series temporales para capturar momentum de temporada
- Ampliar el dataset a todas las temporadas disponibles (1950–2024 completo)

Semanas del proyecto

Semana	Notebook	Contenido
1	`01_exploracion_inicial`	Carga, inspección, primeras visualizaciones
2	`02_api_y_dataset_maestro`	API Jolpica-F1, joins, features derivadas
3	`03_limpieza`	Nulos, outliers, normalización, tipos
4	`04_eda`	Correlaciones, hipótesis, heatmaps
5	`05_modelos_baseline`	Regresión lineal y logística
6	`06_modelos_avanzados`	Random Forest y XGBoost
7	**`07_interpretabilidad_conclusiones`**	**SHAP, PDP, errores, conclusiones**